

Лабораторна робота 02

RAG з нуля і контрольований ablation

Курс «Прикладні ШІ-агенти та системи на основі великих мовних моделей» · 2026/2027

Лекції	6–8
Аудиторний час	4 год; самостійна робота 8 год
Формат	індивідуально
Вартість	0 грн: локальна модель або stub, платні API не потрібні

Мета

Побудувати локальний пошуковий пайплайн над відкритими документами і експериментально показати, що дає кожна зміна конфігурації, вимірюючи це на одному наборі запитів із ручною розміткою.

Теоретичні відомості

Retrieval-augmented generation з'єднує пошук по зовнішньому сховищу з генерацією: модель отримує знайдені фрагменти в контексті і будує відповідь із них, а не з ваг [1]. Оцінювати таку систему як одне ціле марно, бо ламаються дві різні речі: пошук може не знайти потрібний фрагмент, або знайти, а модель його проігнорує. Тому retriever і generator міряються окремо [2, 7].

Одиницею пошуку є не документ, а фрагмент, придатний як контекст. Разом із текстом фрагмента зберігаються метадані: джерело, розділ, дата, права доступу. Без них неможливі ні фільтрація, ні цитування, ні видалення застарілого.

Розріджений пошук зіставляє слова. Оцінка BM25 зростає з частотою терміна в документі та його рідкісністю в колекції і нормується на довжину:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgl}})}$$

де $f(t, d)$ — частота терміна, $|d|$ — довжина документа, avgl — середня довжина, k_1 керує насиченням частоти, b — силою нормування на довжину. Такий пошук сильний на точних термінах, кодах і назвах, і слабкий на перефразуванні.

Щільний пошук зіставляє змісти: запит і фрагмент переводяться в вектори однією моделлю, близькість міряється косинусом кута. Порівнювати косинуси векторів різних моделей не можна — це різні простори. Повний перебір по мільйону векторів не масштабується, тому будується наближений індекс. У HNSW параметр M задає кількість ребер вузла, `efConstruction` — ширину пошуку при побудові, `efSearch` — при запиті; зростання `efSearch` підвищує повноту і затримку одночасно [4].

Оскільки два підходи помиляються по-різному, їхні результати об'єднують. Reciprocal Rank Fusion не потребує узгодження шкал, бо працює з рангами:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

де R — множина списків видачі, $\text{rank}_r(d)$ — позиція документа в списку r . Константа k пригнічує вплив перших позицій і робить злиття стійким [5].

Другим етапом ставлять реранкер. Бієнкодер кодує запит і документ окремо, тому вектори документів обчислюються заздалегідь; кросенкодер подає пару на вхід разом і тому точніший, але вимагає прогону моделі на кожну пару і застосовний лише до кількох десятків кандидатів [2].

Якість пошуку міряють $\text{Recall}@k$, MRR і $\text{nDCG}@k$; якість відповіді — часткою тверджень, підтверджених наданим контекстом (*faithfulness*), і повнотою та точністю самого контексту [7]. Головне правило експерименту: за один прогін змінюється рівно одна річ, і всі конфігурації міряються на тому самому наборі запитів.

Порівнюють не два окремі інтервали, а інтервал *різниці*. Дві конфігурації, виміряні на тому самому наборі, дають парні спостереження: для кожного запиту рахується $\text{Recall}@5$ обох конфігурацій і їхня різниця, а інтервал будується вже для середньої різниці — наприклад, парним *bootstrap* на 10 000 ресемплів. Дисперсія парної різниці менша, ніж у кожної оцінки окремо, тому такий тест бачить менші ефекти. Висновок «інтервали перекриваються, отже різниці немає» некоректний: перекриття маргінальних інтервалів не є критерієм. Окремо врахуйте кластери: десять запитів до одного документа — це одне спостереження з десятима гранями, тому ресемпл робиться за документами. Різниця в кілька запитів на наборі з шістдесяти майже напевно лежить усередині інтервалу різниці й нічого не доводить.

Хід роботи

Позначка *ауд.* — крок виконується на занятті, *сам.* — самостійно в межах СРС. Збирання корпусу і ручна розмітка 60 запитів свідомо винесені на домашній етап: на парі на них часу немає. Аудиторна частина розрахована на 4 години.

1. **Корпус і ліцензії (40 хв, сам.).** 30–100 документів однієї предметної області. *Перевірка:* для кожного джерела в таблиці стоїть ліцензія. Документ без зрозумілої ліцензії до корпусу не потрапляє.
2. **Ingestion (60 хв, ауд.).** Парсинг, очищення, чанкінг, метадані. *Перевірка, яка коштує хвилину і рятує день:* узяти точну фразу з паперового документа і пошукати її серед чанків. Не знайшли — далі йти немає сенсу.
3. **Розмічений набір (60 хв, сам.).** 60+ запитів із вручну позначеними правильними чанками. **Робиться до експериментів і більше не змінюється.** *Перевірка:* запити взяті з того, як питали б реальні користувачі, а не придумані під наявний індекс.
4. **Два індекси (40 хв, ауд.).** Розріджений і щільний. *Перевірка:* для щільного — вектори нормалізовані, і метрика індексу відповідає тому, що ви вважаєте косинусом.
5. **Чотири конфігурації (60 хв, ауд.).** BM25; щільний; гібрид через RRF; гібрид із реранкером. *Змінювати рівно одну річ за раз* — інакше приріст нема кому приписати.
6. **Довірчі інтервали (30 хв, ауд.).** Для кожної *пари* конфігурацій порахувати інтервал парної різниці per-query Recall@5 (bootstrap за документами). *Перевірка:* явно назвати пари, різниця яких накриває нуль. Це головний результат роботи, а не невдача.
7. **Власний ablation (40 хв, сам.).** Одна зміна за вибором, з тим самим набором.
8. **Генерація з цитуванням (40 хв, ауд.).** *Перевірка:* при порожньому пошуку система відмовляється, а не вигадує. Цей випадок має бути в наборі.
9. **Звіт (40 хв, сам.).** Таблиця конфігурацій з інтервалами і висновком.

Порядок кроків 3 і 4 не переставляти: набір, складений після того, як побачили видачу індексу, міряє індекс, а не задачу.

Завдання

1. Зібрати корпус із 30–100 відкритих документів однієї предметної області з явно зазначеною ліцензією. Джерела і ліцензії звести в таблицю.
2. Реалізувати ingestion: парсинг, очищення, чанкінг із перекриттям, збереження метаданих (джерело, сторінка або розділ, дата). Обґрунтувати обраний розмір чанка, порахувати кількість чанків і розмір індексу в мегабайтах.
3. Побудувати два індекси: розріджений (rank_bm25 або аналог) і щільний (sentence-transformers з невеликою багатомовною моделлю, FAISS або sqlite-vec). Дотриматися контракту обраної моделі: для сімейства E5 обов'язкові префікси query: і passage:, для косинуса у FAISS — нормалізація векторів разом із METRIC_INNER_PRODUCT, інакше індекс рахує евклідову відстань, а не те, що ви вважаєте косинусом.
4. Скласти набір із не менш ніж 60 запитів природною мовою і вручну розмітити для кожного правильні чанки. Набір фіксується до початку експериментів і більше не змінюється.

5. Виміряти чотири конфігурації на цьому наборі: лише BM25; лише щільний пошук; гібрид через RRF; гібрид із кросенкодерним ранкером top-20 → top-5. Для кожної навести Recall@5, MRR і затримку p50.
6. Для кожної пари конфігурацій порахувати 95 % довірчий інтервал *парної різниці* per-query Recall@5 (парний bootstrap, ресемпл за документами) і явно сказати, які пари розрізняються, а в яких інтервал накриває нуль. Маргінальні інтервали окремих конфігурацій навести можна, але висновок з їх перекриття не робиться.
7. Провести один додатковий ablation за власним вибором: розмір чанка, перекриття, кількість кандидатів або efSearch. Змінити рівно один параметр.
8. Додати генерацію відповіді з цитуванням: кожне твердження супроводжується вказівником на чанк, а за відсутності релевантного контексту система відмовляється відповідати. Перевірити на 10 запитах, серед яких два свідомо поза корпусом.
9. Здати: код і README, набір запитів із розміткою, таблицю чотирьох конфігурацій із інтервалами парних різниць, результат додаткового ablation, 10 прикладів відповідей із цитатами і висновок до 200 слів про те, яка зміна дала найбільший приріст на одиницю затримки.

Контрольні запитання

1. Чому оцінювання retriever і generator розділяють, і що показує кожна з двох оцінок?
2. Що станеться з BM25, якщо параметр b дорівнює нулю?
3. Чому косинусна близькість між векторами двох різних моделей не має сенсу?
4. Що саме змінює efSearch у HNSW і чим за це платять?
5. Навіщо в RRF константа 60 і що буде, якщо взяти 1?
6. Чому кросенкодер не застосовують до всієї бази?
7. Recall@5 зріс, а якість відповідей — ні. Які причини можливі?
8. Як відрізнити випадок «чанк не знайдено» від випадку «чанк знайдено, але зігноровано»?
9. Чому набір запитів фіксується до експериментів, а не складається за їх результатами?
10. Коли пошук взагалі не потрібен і документи можна подати в контекст цілком?

Література

1. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
2. Gao Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997, 2024.
3. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. FnTIR, 2009.
4. Malkov Yu., Yashunin D. Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs. TPAMI, 2018.
5. Cormack G., Clarke C., Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR, 2009.

6. Thakur N. et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS, 2021.
7. Es S. et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217, 2023.
8. Jurafsky D., Martin J. H. Speech and Language Processing, 3rd ed. draft, 2026. Розділи 5, 11.
9. Anthropic. Introducing Contextual Retrieval, 2024.
10. Facebook Research. FAISS: Guidelines to choose an index.

Походження шаблону

Макет адаптовано з Laboratory report template for Basic Chemistry at Stockholm University, Viktor Bengtsson, LPPL 1.3c: article, Times-подібна гарнітура, широкі поля, fancyhdr, booktabs.